PyPTO：Tensor的算子编程范式

作者：尹杰

时间：2026年3月

https://gitcode.com/cann/pypto

CANN

目录

Part 1 编程原理

Part 2 编程界面

Part 3 功能调试

Part 4 性能调优

CANN

PyPTO：Tensor的算子编程范式

PyPTO 是什么?

- **本质：** 面向 AI 加速器的高性能编程框架
- **目的：** 简化算子开发流程，同时保持高性能计算能力
- **范式：** 创新的 PTO 编程
- **理念：** 基于 Tensor/Tile 的编程模型

PyPTO 的定位

- 不仅限于单个算子的开发，更强调**复杂融合算子，甚至于整个模型网络**的开发能力
- 已有编程框架的**补充而非替换**，特别适合需要深度性能优化和细粒度控制的场景

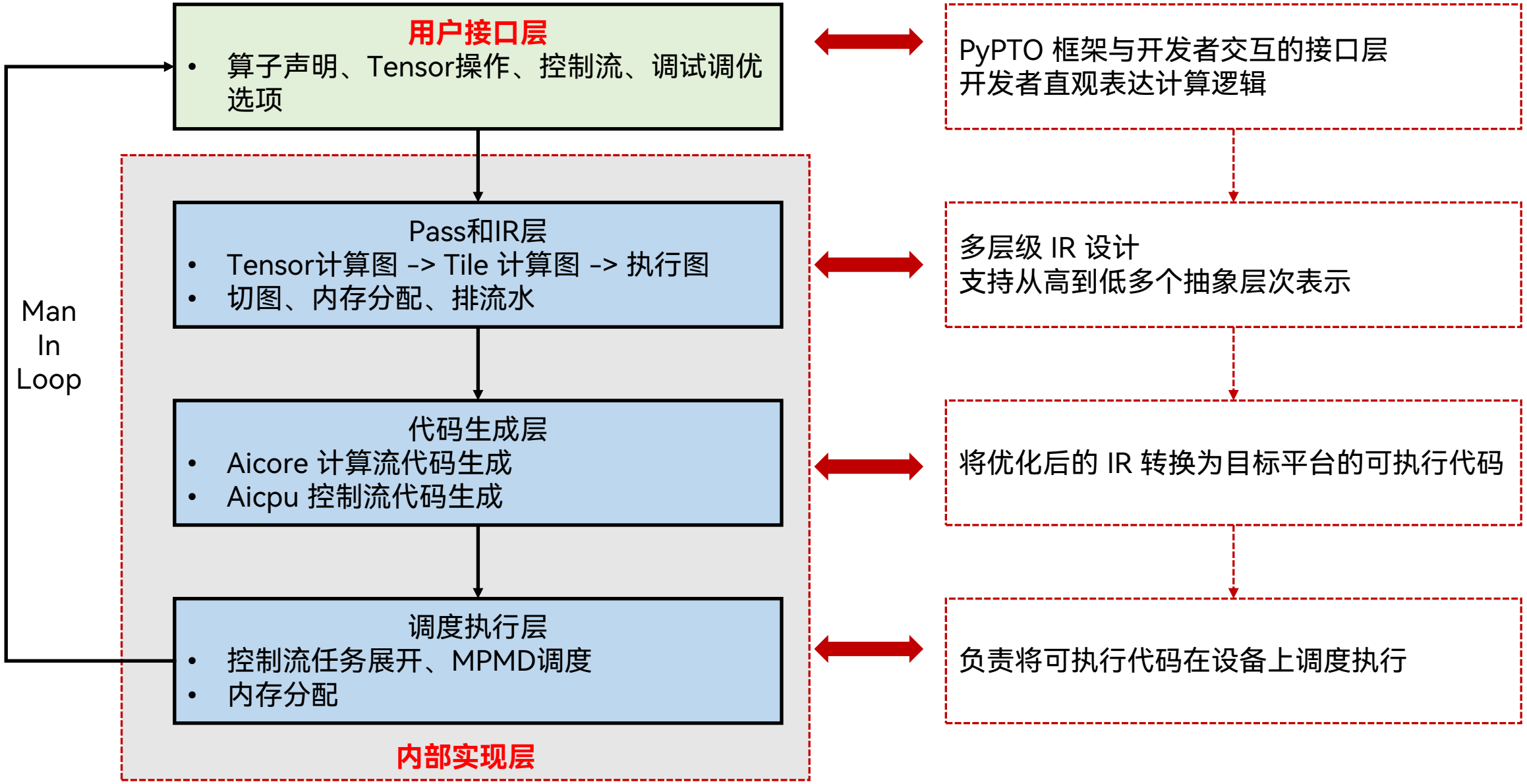
https://gitcode.com/cann/pypto

PyPTO

- **Tensor/Tile 编程**
  以 Tensor/Tile 为粒度编程
  屏蔽不必要的优化细节
- **Python 原生**
  提供 Python 原生接口
  降低用户开发门槛
- **跨代兼容**
  通过底层 Tile 指令集
  实现跨代兼容
- **白盒化调优**
  充分发挥专家经验
  编译流程白盒化、可视化

CANN

Tensor 编程和 Tile 优化原理



本章小结

- **本章对 PyPTO 编程框架进行了概述**
  - PyPTO 是面向 AI 加速器的高性能编程框架
  - PyPTO 的 Tensor 编程用户主要关心计算表达，通过框架内的多层次处理来实现高性能

目录

Part 1 编程原理

Part 2 编程界面

Part 3 功能调试

Part 4 性能调优

CANN

编程界面概览

- PyPTO算子编程界面包括算子声明、Tensor 创建和计算、动态控制流、标量计算
- 算子调用方式：PyTorch集成

② Tensor 创建和计算

① 算子声明

③ 动态控制流

④ 标量计算

```
import pypto

def softmax_core(x: pypto.Tensor) -> pypto.Tensor:
    row_max = pypto.amax(x, dim=-1, keepdim=True)  # 计算行最大值
    sub = x - row_max                              # 值归一化
    exp = pypto.exp(sub)                           # 指数运算
    esum = pypto.sum(exp, dim=-1, keepdim=True)    # 求和
    return exp / esum                              # 概率归一化

@pypto.jit
def dynamic_softmax(input_tensor : pypto.Tensor, output_tensor: pypto.Tensor):
    # 获取动态维度
    batch_size = input_tensor.shape[0]
    tile_size = pypto.symbolic_scalar(64)
    loop_count = batch_size // tile_size

    # 循环处理
    for idx in pypto.loop(0, loop_count, 1, name="LOOP_BATCH"):
        offset = idx * tile_size
        end = (idx + 1) * tile_size
        # 提取当前 Tile
        x_view = input_tensor[offset:end, :]
        # 计算 Softmax
        softmax_out = softmax_core(x_view)
        # 组装结果
        output_tensor[offset:end, :] = softmax_out

# 执行
dynamic_softmax(input_tensor, output_tensor)
```

https://gitcode.com/cann/pypto

CANN

算子声明

- 使用方式：Kernel 入口函数通过 @pypto.jit 装饰器定义，在第一次调用时会进行JIT编译。
- 实现原理：在JIT编译时，框架会通过遍历执行计算的方式，为每一条路径构建一个 Tensor 的计算图，并将其转换为 IR 传递给后续的 pass。

```
import pypto
# 1. 配置 Tiling(可选，框架会提供默认值)
pypto.set_vec_tile_shapes(64)

# 2. 定义计算函数
@pypto.jit
def my_operator(a: pypto.Tensor, b: pypto.Tensor, output: pypto.Tensor):
    # Tensor 操作
    result = a + b  # 或使用 pypto.add(a, b)
    output[:] = result

# 3. 执行
my_operator(tensor_a, tensor_b, output_tensor)
```

https://gitcode.com/cann/pypto

CANN

Tensor 创建和计算

- Tensor 的创建
  - 使用方式：
    - ✓ 使用NZ格式创建一个Tensor

```
tensor = pypto.tensor([-1, 32], pypto.DT_FP16, "nz_tensor", pypto.TileOpFormat.TILEOP_NZ)
```

    - ✓ 动态维度

```
tensor = pypto.tensor([-1, 32], pypto.DT_FP16, "dynamic")
b = pypto.symbolic_scalar(tensor_shape[0])
```

  - 实现原理：对应在 IR 中一个逻辑 Tensor 节点
- Tensor 的计算
  - 使用方式
    - ✓ 算术运算，例如加法

```
result = a + b
```

    - ✓ 数学函数，例如指数和对数

```
result = pypto.exp(x)     # e ** x
```

    - ✓ 激活函数，例如Sigmoid

```
result = pypto.sigmoid(x)  # 1 / (1 + exp(-x))
result = x.sigmoid()
```

    - ✓ 比较函数，例如最大值和最小值

```
result = pypto.maximum(a, b)  # 逐元素最大值
result = pypto.minimum(a, b)  # 逐元素最小值
```

    - ✓ 规约操作，例如求和

```
result = pypto.sum(x, dim=-1, keepdim=False)
```

    - ✓ 矩阵运算，例如C = A @ B，其中 A: [M, K], B: [K, N], C: [M, N]

```
result = pypto.matmul(A, B, out_dtype=pypto.DT_BF16)
```

    - ✓ 视图操作

```
view = pypto.view(tensor, view_shape, offset, valid_shape)
```

    - ✓ 组装操作

```
pypto.assemble(source, offsets=[10, 20], dest)
```

    - ✓ 类型转换

```
result = pypto.cast(tensor, pypto.DT_FP32, mode=pypto.CastMode.CAST_NONE)
```

    - ✓ 维度

```
reshaped = pypto.reshape(tensor, [new_shape])
transposed = pypto.transpose(tensor, dim0=0, dim1=1)
```

  - 实现原理：对应在 Tensor 的计算图中的一个计算 op 节点

CANN

动态控制流

- 循环：在静态循环的基础上，支持编译时无法静态确定的动态循环。
  - 使用方式：通过 pypto.loop() 定义一个动态循环
  - 实现原理：对应 IR 中的一个循环，循环按需生成独立的静态子图

```
# 处理动态维度数据
tile_size = pypto.symbolic_scalar(64)
loop_count = dynamic_shape / tile_size

for idx in pypto.loop(0, loop_count, 1, name="LOOP_BATCH"):
    offset = idx * tile_size
    end = (idx + 1) * tile_size
    input_view = input_tensor[offset:end, :]
    output_tensor[offset:end, :] = process_tile(input_view)
```

- 分支：在静态分支的基础上，支持编译时无法静态确定的动态分支。
  - 使用方式：通过 pypto.cond() 定义一个动态分支
  - 实现原理：对应 IR 中的一个分支，每一条路径生成一个独立的静态子图

```
for idx in pypto.loop(b_loop):
    t3_sub = t0_sub + t1_sub
    if pypto.cond(idx < 2):  # 动态条件判断
        t2[b_offset:b_offset_end, ...] = t3_sub + 1
    else:
        t2[b_offset:b_offset_end, ...] = t3_sub
```

CANN

标量计算

- PyPTO支持普通的标量计算，即立即数计算
  - 使用方式：普通的标量操作计算
  - 实现原理：静态结果在编译时对应 IR 中的立即数

    a = b + 1

- PyPTO支持符号化标量(SymbolicScalar），用于支持动态Shape Tensor的表达和处理，使得框架可以在编译时进行Shape推断和优化
  - 使用方式：普通的标量操作计算
  - 实现原理：对应 IR 中的动态标量计算

```
# 创建动态Shape Tensor
tensor = pypto.tensor([-1, 32], pypto.DT_FP16, "dynamic")

# 获取动态维度的符号化标量，在运行时获取具体数值
b = pypto.symbolic_scalar(tensor_shape[0])
```

CANN

算子调用方式：PyTorch 集成

- 使用方式：通过 pypto.from_toch 将 torch tensor 转为 pypto tensor，从而可以作为算子的输入和输出参数传进来并计算。
- 实现原理：在编译生成 kernel 后，通过 rtKernelLaunch 类接口，将输入输出 tensor 的地址传给设备侧，执行kernel后返回。

```
import pypto
import torch

@pypto.jit
def my_operator(x: pypto.Tensor, output: pypto.Tensor):
    result = pypto.matmul(x, weight)
    output[:] = result

# 使用 PyTorch Tensor
input_torch = torch.randn(32, 128, device='npu')
output_torch = torch.zeros(32, 64, device='npu')

# 转换为 PyPTO Tensor
pto_input = pypto.from_torch(input_torch, "INPUT")
pto_output = pypto.from_torch(output_torch, "OUTPUT")

# 执行
my_operator(pto_input, pto_output)
```

本章小结

- **本章对 PyPTO 的 Tensor 编程方式做了总结**
  - 通过 @pypto.jit 来定义算子
  - 通过 pypto.Tensor 创建 Tensor，通过 pypto.<op> 实现算子计算
  - 通过 pypto.loop 实现动态循环，通过 pypto.cond 实现动态分支
  - 通过 pypto.symbolic_scalar 实现动态标量
  - 通过 pypto.from_torch 将 torch Tensor 转为 PyPTO Tensor，通过将其传给定义的算子来实现在昇腾硬件上的调用

目录

Part 1 编程原理

Part 2 编程界面

Part 3 功能调试

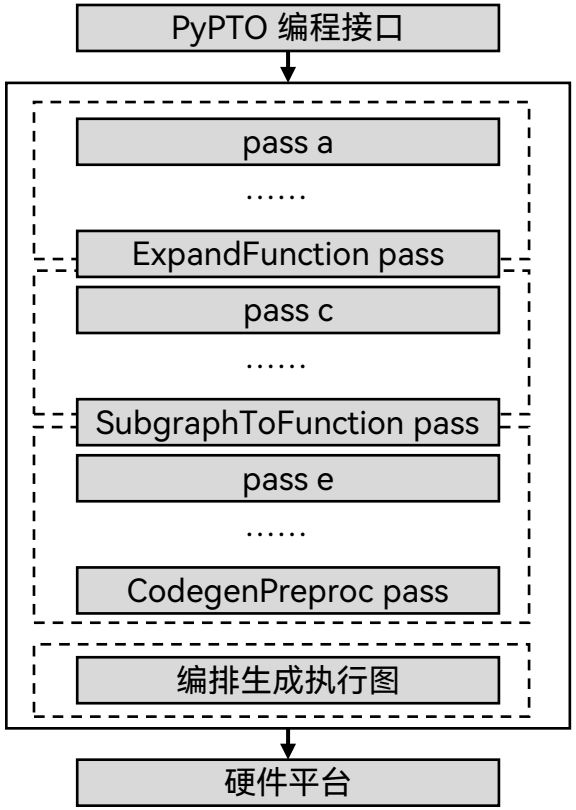
Part 4 性能调优

CANN

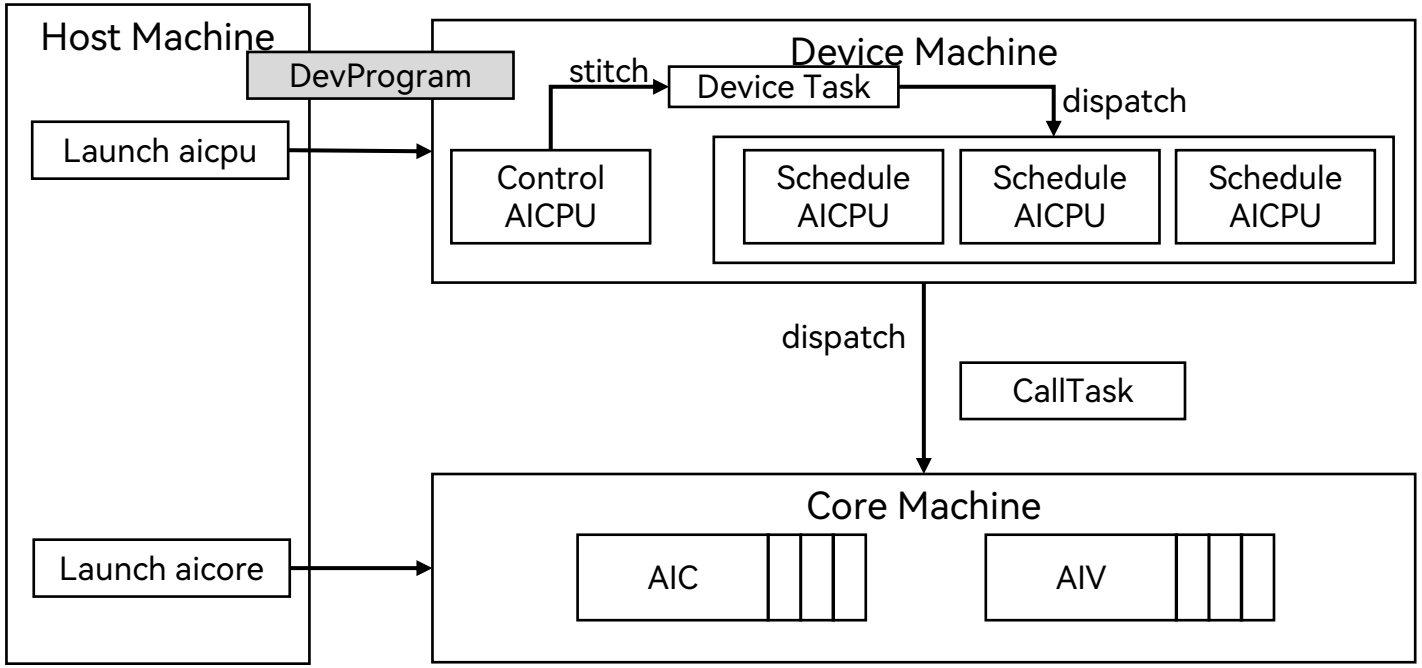
PyPTO框架运行流程

- 编程界面：当开发者完成算子开发后，框架会生成由Tensor和Operation交错连接的图结构，即计算图。
- 编译时：该计算图经过PyPTO编译优化流程，最终生成可在昇腾硬件环境中运行的可执行代码
- 运行时：运行时通过框架通过加载编译得到的可执行代码，来实现实际的计算任务。

- **Tensor Graph 阶段**
实现和硬件无关的图优化，包括冗余操作消除等
- **Tile Graph 阶段**
根据Tile Graph进行Tile展开，实现Tile 级别的优化，包括内存分配等
- **Block Graph 阶段**
切分生成计算子图，进行Block级别的优化
- **Execute Graph 阶段**
整合计算子图信息，生成最终执行图



计算图编译流程

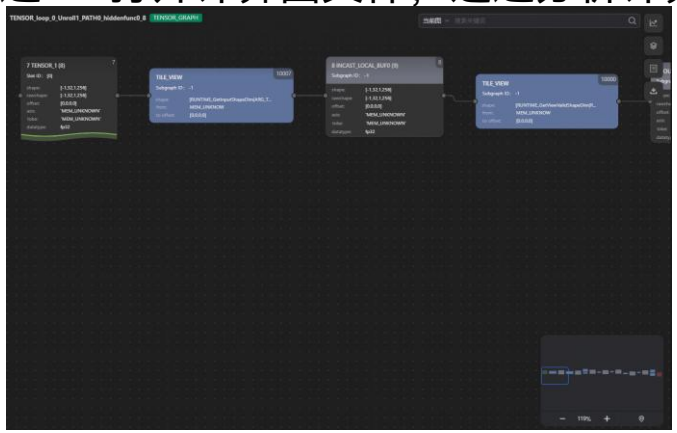


执行态运行示意图

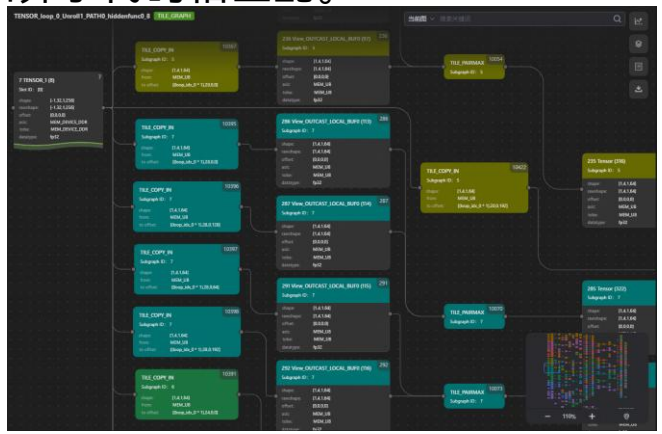
基于计算图的算子正确性调试流程

- 如果图编译或图执行流程中出现错误或结果不符合预期，可以启用调试模式，生成不同阶段的计算图文件
  1. 开启开关：@pypto.jit(debug_options={"compile_debug_mode": 1})
  2. 执行用例：python3 examples/02_intermediate/operators/softmax/softmax.py
  3. 计算图结果输出：在${work_path}/output/output_*/目录(*代表时间戳）下生成不同阶段的计算图文件(.json 格式）。
  4. 计算图可视化：通过IDE打开计算图文件，通过分析计算图中计算节点之间关系是否和算子代码相匹配。

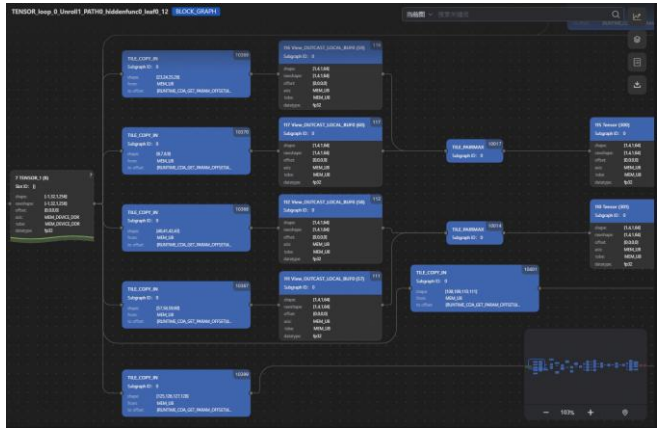
- Tensor 计算图（Tensor Graph）



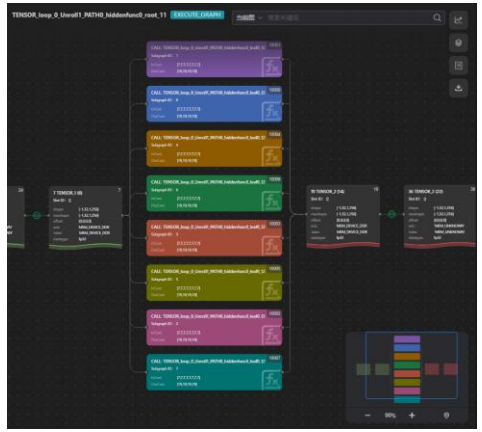
- Tile 计算图（Tile Graph）



- 核内计算图（Block Graph）



- 执行图（Execute Graph）



本章小结

- **本章对 PyPTO 的 Tensor 问题定位方式做了介绍**
  - PyPTO框架运行流程分为编译时和运行时
  - PyPTO 可以基于计算图调试算子的正确性

目录

Part 1 编程原理

Part 2 编程界面

Part 3 功能调试

Part 4 性能调优

CANN

性能调优工具：PyPTO Toolkit

- 在完成功能后，开发者可以利用PyPTO Toolkit查看对应算子的泳道图，直观展示计算图的实际调度与执行过程。
- 泳道图配置
  1. 采集泳道图数据：通过给@pypto.jit装饰器的入参debug_options配置图执行阶段调试开关启动性能数据采集功能。
     @pypto.jit(debug_options={"runtime_debug_mode": 1})
  2. 执行用例：python3 examples/02_intermediate/operators/softmax/softmax.py
  3. 生成泳道图json文件：在当前工作目录的output/output_时间戳目录下生成merged_swimlane.json，该文件为泳道图数据文件。可以通过IDE打开来查看泳道图。

- 常见调优方式
  - 开箱性能调优：算子LOOP调优，Tile切分调优
    - ***Tips：在算子开发阶段需要注意循环的使用以及选择合适的Tilesize参数***
  - 深度调优：执行图的静态合并、运行时任务的划分与调整

CANN

开箱性能：Loop优化

- 设计原理：不同循环之间的静态图不能合并，静态图合并是PyPTO优化性能的关键手段
- 核心原则：**增加每次迭代内静态图的大小，减少静态图个数**

✓ **场景1：尽可能合并 loop：**检查算子代码是否有可以合并的loop块，应当将其合并以增大静态图。

```
bsz = x1.shape[0]
for b_idx in pypto.loop(bsz, name="LOOP_1", idx_name="b_idx"):
    out_1 = OP_1(x1[b_idx, :], y)
for b_idx in pypto.loop(bsz, name="LOOP_2", idx_name="b_idx"):
    out_2 = OP_2(x2[b_idx, :], y)
```

✓ **场景2：动态轴范围较广时使用 loop_unroll:** loop_unroll功能与loop类似，在其基础上增加了unroll_list参数支持多个展开方式。

```
"""
input：A， shape:[-1, 64]
output：B， shape:[-1, 64]
-1：表示动态shape
"""
.....
for b, k in pypto.loop_unroll(A.shape[0] // 64, unroll_list=[64, 16, 4], name="A", idx_name='b'):
  ### 支持在不同的展开档位设置不同调优参数
  if k <= 16:
    pypto.set_vec_tile_shapes(16, 64)
  else :
    pypto.set_vec_tile_shapes(64, 64)

  tile_a = A[b * 64:(b + k) * 64, :]  #
  tile_a = tile_a + 2
  B[b * 64:, :] = tile_a
```

https://gitcode.com/cann/pypto

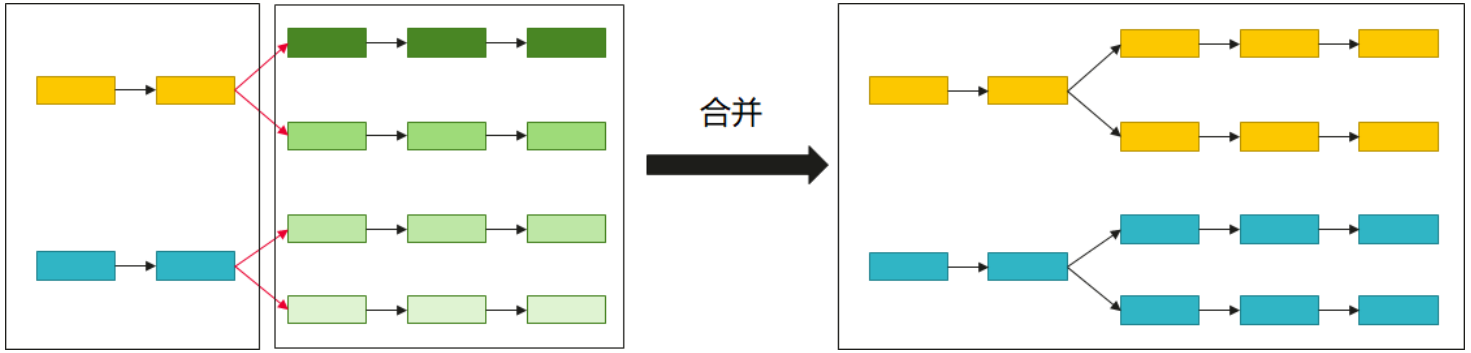
CANN

开箱性能：Tile切分

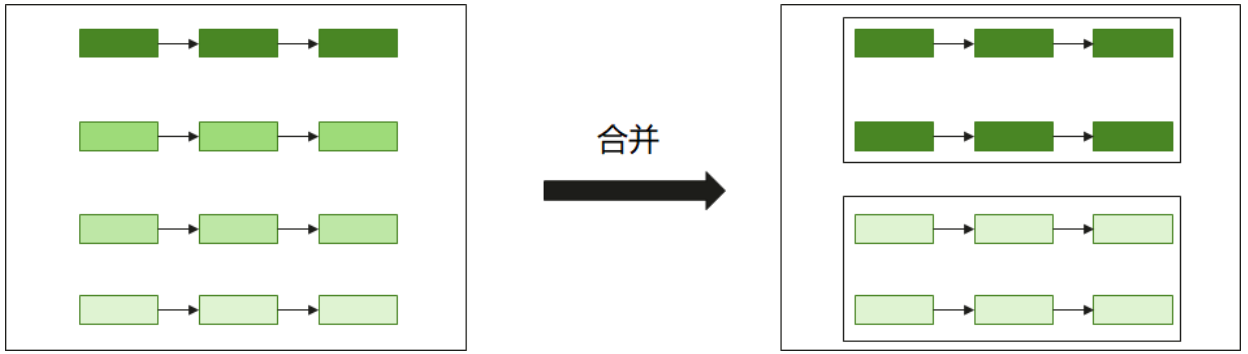
- 设计原理：Tilesize 切分大小直接决定了算子切分后的任务数量，从而决定了实际执行时的分核数、计算轮次
- 核心原则：优化Tile配置。
- Cube 计算(矩阵乘)
  - Cube配置原则：针对矩阵运算场景，基于A2/A3平台，以A、B矩阵均为BF16或FP16类型为例，满足Buffer空间约束的推荐Tile配置为：
  - 建议的默认参数：Cube的相关计算建议采用如下的TileShape：
  - pypto.set_cube_tile_shapes([128, 128], [64, 256], [256, 256])
  - pypto.set_cube_tile_shapes([256, 256], [64, 256], [128, 128])
  - pypto.set_cube_tile_shapes([128, 128], [128, 512], [128, 128])
- Vector 计算
  - Vector配置原则：针对向量运算场景，我们应该按照OP和芯片UB大小来确定合适的Tilesize。
  - 建议的默认参数：Vector的相关计算建议采用如下的TileShape：
  - pypto.set_vec_tile_shapes(64, 512)

深度调优：执行图的静态合并

- **合图设计原理：**将计算图中多个逻辑上独立的Operation合并为一个逻辑子图，并由该子图最终生成一个计算Kernel过程。
- **深度方向合图：**深度方向合图基于计算图中的生产者-消费者关系，沿数据依赖路径将前后相邻的 Operation 进行融合。
  - 优化原理：该方式通过消除中间结果的写回操作，直接优化数据流路径，使原本受限于带宽的算子链得以在单个内核内一气呵成地完成计算。



- **广度方向合图：**广度方向合图针对计算图中处于同一层级、可并行执行的 Operation。
  - 优化原理：通过将多个并行 Operation 合并到同一 Kernel 中执行以增强单次 Kernel 的计算规模。

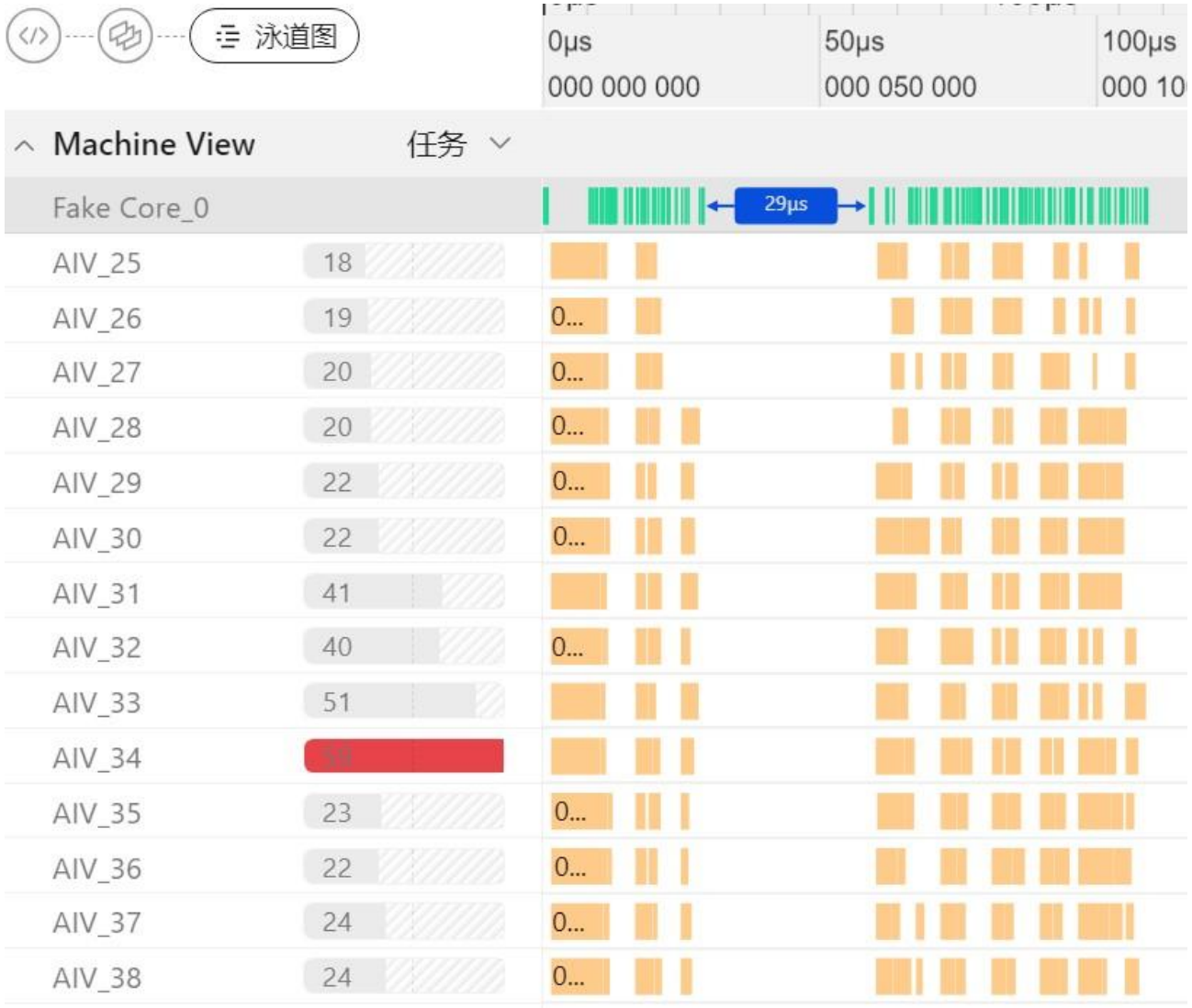


https://gitcode.com/cann/pypto

CANN

深度调优：运行时任务的动态合并

- Stitch 是一种静态图运行时并行的方法。
- **设计原理：**Stitch配置决定了多少个静态图被并行下发调度。
- **调优原则：**需要结合泳道图数据调整如上参数，使总耗时减少。
  - Stitch设置较大时workspace也会增加，大量任务并行，可能导致L2命中率较低。



更多调试调优方式详见文档和《高效编程指南》

- 高效编程指南：https://pypto.gitcode.com/tutorials/debug/performance.html

本章小结

- **本章对 PyPTO 的 Tensor 性能调优方式做了介绍**
  - PyPTO 性能调优主要通过 PyPTO Toolkit 展示的泳道图查看
  - 开箱性能调试方法主要包括 Loop 优化和 Tile 切分
  - 深度调优主要包括执行图的静态合并，以及运行时任务的动态合并
  - 更多调优方法详见《高效编程指南》（https://pypto.gitcode.com/tutorials/debug/performance.html）

扫码获取《高效编程指南》

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann/pypto